

COVID-19 Chest X-Ray Classification with AI

A complete deep learning pipeline for automated diagnosis using transfer learning and computer vision

Objective

Classify chest X-ray images into four diagnostic categories using MobileNetV2 transfer learning

Pipeline Overview

01

Environment Setup

02

Data Ingestion

03

Preprocessing

04

Model Training

05

Evaluation

Dataset

COVID-19 Radiography Database — Kaggle

Base Model

MobileNetV2 pretrained on ImageNet

Classes

4-class: COVID, Normal, Lung Opacity, Viral Pneumonia

Framework

TensorFlow 2.x / Keras with tf.data pipelines

Problem Statement & Proposed Solution

The COVID-19 Diagnostic Crisis

Rapid and accurate diagnosis of COVID-19 was a massive healthcare challenge during the pandemic. Overwhelmed medical systems and limited PCR testing resources created critical bottlenecks in patient triage and treatment allocation.

- Radiologists faced unprecedented volume of chest X-ray reviews
- Manual interpretation is time-consuming and subject to fatigue-induced error
- Delayed diagnosis directly impacted patient outcomes and resource planning

COVID

Label 0 — Ground-glass opacities, bilateral infiltrates

Lung Opacity

Label 2 — Non-COVID pulmonary consolidation patterns

The AI-Driven Solution

Building a Computer Vision model to predict and classify chest X-ray images based on previously diagnosed medical datasets — automating the triage process with deep learning.

- Transfer learning from ImageNet-pretrained MobileNetV2 reduces training data requirements
- Standardized preprocessing pipeline ensures consistent model input quality
- 4-class output enables nuanced differential diagnosis beyond binary COVID detection

Normal

Label 1 — Clear lung fields, no pathological findings

Viral Pneumonia

Label 3 — Viral infection patterns distinct from COVID-19

Environment Setup & Essential Libraries

Install and import all core frameworks before beginning the pipeline. Run the pip install command first in a fresh environment.

```
!pip install opencv-python tensorflow
```

```
import kagglehub
import cv2
import os
import psutil
import numpy as np
import random
import keras
import tensorflow as tf
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

kagglehub

Programmatic dataset
download from Kaggle API

cv2 (OpenCV)

Image reading, color
conversion, and resizing

numpy

Matrix manipulation and array
operations

tensorflow / keras

Deep learning model building,
training, and evaluation

sklearn

Train/test splitting and classification metrics reporting

 Run `psutil` to monitor RAM usage during in-memory loading — large datasets can exhaust system memory without the `tf.data` pipeline approach.

Dataset Path Handling & Setup

Download & Directory Configuration

```
# 1. Download & Main Directory
path = kagglehub.dataset_download(
    "tawsifurrahman/covid19-radiography-database"
)
BASE_DIR = path
print("Path to dataset files:", BASE_DIR)

DATASET_DIR = os.path.join(
    path, "COVID-19_Radiography_Dataset"
)
```

What This Block Does

kagglehub.dataset_download() authenticates with the Kaggle API and pulls the COVID-19 Radiography Database directly into the runtime environment — no manual download required.

- **BASE_DIR** — root path returned by kagglehub, points to the downloaded archive location
- **DATASET_DIR** — navigates one level deeper into the named dataset subfolder containing the four class directories
- **os.path.join()** — constructs OS-agnostic paths, ensuring compatibility across Linux, macOS, and Windows runtimes

❏ Kaggle API credentials (`kaggle.json`) must be configured in the environment before calling `dataset_download()`.

Label Categorization & Class Directories

Category & Path Definitions

```
# 1. Categories
categories = [
    'COVID',
    'Lung_Opacity',
    'Normal',
    'Viral Pneumonia'
]

# 2. Directories
COVID_DIR = os.path.join(
    DATASET_DIR, "COVID", "images"
)
NORMAL_DIR = os.path.join(
    DATASET_DIR, "Normal", "images"
)
LUNG_OPACITY_DIR = os.path.join(
    DATASET_DIR, "Lung_Opacity", "images"
)
VIRAL_PNEUMONIA_DIR = os.path.join(
    DATASET_DIR, "Viral Pneumonia", "images"
)
```

Label-to-Directory Mapping

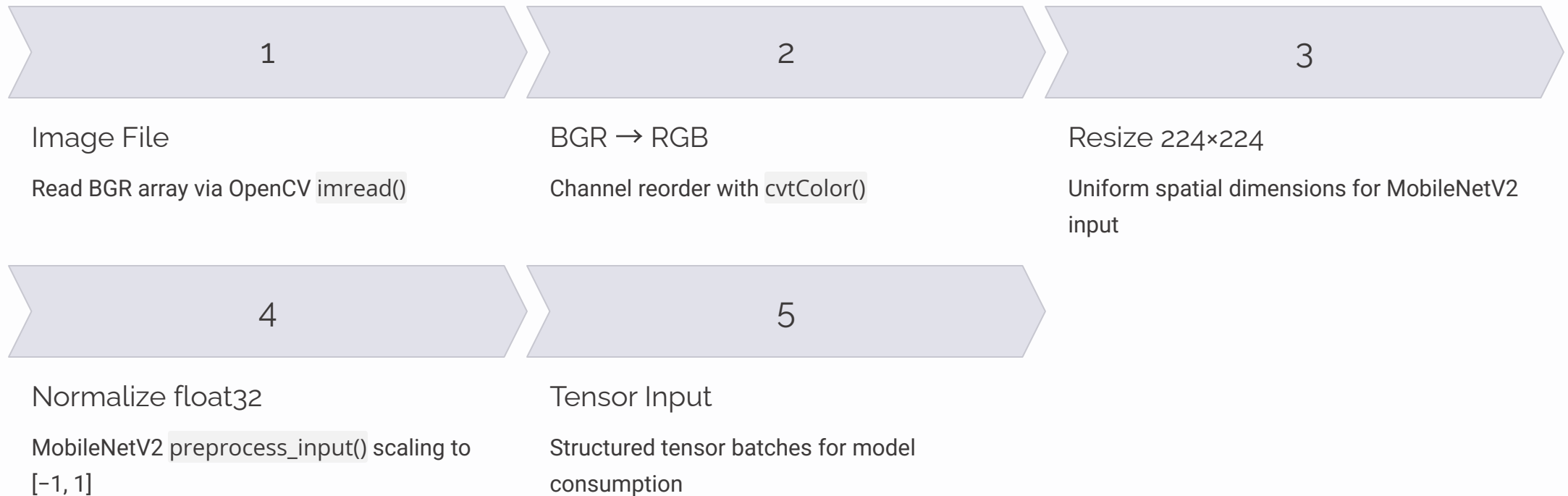
Each class directory contains only the raw image files for that diagnostic category. The `images/` subdirectory is consistent across all four class folders in the Radiography Database structure.

Label	Category	Directory Variable
0	COVID	COVID_DIR
1	Normal	NORMAL_DIR
2	Lung Opacity	LUNG_OPACITY_DIR
3	Viral Pneumonia	VIRAL_PNEUMONIA_DIR

Integer labels are required for `sparse_categorical_crossentropy` loss and `classification_report` output downstream.

Data Preprocessing Pipeline Overview

Every raw X-ray image passes through a deterministic six-stage transformation before reaching the model. Consistency across all samples is critical for stable gradient updates during training.



Why BGR → RGB?

OpenCV reads images in BGR channel order by default. MobileNetV2 was pretrained on ImageNet using RGB ordering. Skipping this conversion corrupts the color channel semantics and degrades transfer learning performance.

Why 224×224?

MobileNetV2's convolutional architecture expects a fixed spatial input of **224 × 224 × 3**. All images — regardless of original resolution — must be resized to this shape before entering the base model.

Why float32 Normalization?

Raw pixel values range from 0–255 (uint8). `preprocess_input()` rescales to `[-1, 1]`, matching the distribution seen during ImageNet pretraining and accelerating gradient convergence.

Image Loader & Preprocessing Function

Loader Function Definition

```
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 64

def load_images_from_folder(folder_path, label):
    images = []
    labels = []

    image_files = [
        f for f in os.listdir(folder_path)
        if f.lower().endswith(('.png', '.jpg', '.jpeg'))
    ]

    for filename in image_files:
        file_path = os.path.join(folder_path, filename)
        image = cv2.imread(file_path)    # reads as BGR
        image = cv2.cvtColor(
            image, cv2.COLOR_BGR2RGB
        )    # convert to RGB
        image = cv2.resize(image, IMAGE_SIZE) # resize
        images.append(image)
        labels.append(label)

    return images, labels
```

Function Breakdown

This utility function encapsulates the full per-image preprocessing sequence into a single reusable call. It accepts a directory path and an integer label, returning two parallel lists ready for array conversion.

1 File Filtering

List comprehension filters only valid image extensions (.png, .jpg, .jpeg), ignoring metadata files and masks present in the dataset

2 Sequential Transform

imread → cvtColor → resize applied per image; results appended to in-memory lists

3 Label Pairing

Each image is paired with its integer label at append time, maintaining index alignment between images[] and labels[]

📌 **BATCH_SIZE = 64** is set here as a global constant used later by both the in-memory and tf.data pipeline approaches.

Dataset Extraction & Label Assignment

Per-Class Loading Calls

```
print("Loading training covid images...")
covid_images, covid_labels = load_images_from_folder(
    COVID_DIR, 0
)

print("Loading training normal images...")
normal_images, normal_labels = load_images_from_folder(
    NORMAL_DIR, 1
)

print("Loading training lung opacity images...")
lung_opacity_images, lung_opacity_labels = \
    load_images_from_folder(LUNG_OPACITY_DIR, 2)

print("Loading training viral pneumonia images...")
viral_pneumonia_images, viral_pneumonia_labels = \
    load_images_from_folder(VIRAL_PNEUMONIA_DIR, 3)
```

Execution Strategy

Each class is loaded independently in sequence. This design isolates loading errors to a single category and makes memory profiling per class straightforward using psutil.

Variable Pair	Label	Source Directory
covid_images, covid_labels	0	COVID_DIR
normal_images, normal_labels	1	NORMAL_DIR
lung_opacity_image s, lung_opacity_labels	2	LUNG_OPACITY_DIR
viral_pneumonia_im ages, viral_pneumonia_la bels	3	VIRAL_PNEUMONIA_DIR

After all four calls complete, concatenate with np.concatenate() and shuffle before splitting into train/validation/test sets using train_test_split().

Data Preprocessing Approaches

Approach 1 — Direct In-Memory Preprocessing (Simple)

```
X_train = preprocess_input(X_train.astype('float32'))
X_test  = preprocess_input(X_test.astype('float32'))
```

Mechanism

Casts entire NumPy arrays to float32 in RAM, then applies MobileNetV2 pixel scaling in a single vectorized operation

Best For

Small datasets that fit comfortably within available system RAM; simplest implementation path

Limitation

Memory-intensive — the full dataset must reside in RAM simultaneously, which is impractical for large collections

Approach 2 — High-Performance tf.data Pipeline (Recommended)

```
import tensorflow as tf
from keras.applications.mobilenet_v2 import preprocess_input

train_dataset = tf.data.Dataset.from_tensor_slices(
    (X_train, y_train)
)
val_dataset = tf.data.Dataset.from_tensor_slices(
    (X_val, y_val)
)

def preprocess_batch(image, label):
    image = preprocess_input(tf.cast(image, tf.float32))
    return image, label

train_dataset = (train_dataset
    .map(preprocess_batch,
        num_parallel_calls=tf.data.AUTOTUNE)
    .shuffle(1000)
    .batch(64)
    .prefetch(tf.data.AUTOTUNE))

val_dataset = (val_dataset
    .map(preprocess_batch,
        num_parallel_calls=tf.data.AUTOTUNE)
    .batch(64)
    .prefetch(tf.data.AUTOTUNE))
```

map()

Parallel CPU preprocessing with AUTOTUNE workers

shuffle()

Randomizes sample order each epoch (buffer=1000)

prefetch()

Overlaps GPU training with CPU data loading

Transfer Learning Model Architecture

Model Definition

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(224, 224, 3)),
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(4, activation="softmax")
])
```

Architecture Rationale

1 MobileNetV2 Base (Frozen)

`include_top=False` removes ImageNet's 1000-class head.
`base_model.trainable = False` freezes all 154 pretrained layers — preserving learned edge, texture, and shape detectors from ImageNet without overwriting them during initial training.

2 GlobalAveragePooling2D

Collapses the spatial feature map output of MobileNetV2 (7×7×1280) into a flat 1280-dimensional vector — reducing parameters versus Flatten while retaining spatial context.

3 Dense Classification Head

Two ReLU dense layers (128 → 64 units) with Dropout(0.3) regularization at both entry and exit points. Final Dense(4, softmax) outputs class probabilities summing to 1.0 across all four diagnostic categories.

- ❑ After initial convergence, unfreeze the top layers of `base_model` and fine-tune with a low learning rate (e.g. 1e-5) to adapt ImageNet features to chest X-ray domain characteristics.

Keras Model Training & Evaluation

A structured walkthrough of model compilation, training with early stopping, and export with full performance evaluation using Keras and TensorFlow.

Compilation

Configure optimizer, loss function, and metrics

Training

Fit with early stopping and validation monitoring

Export & Evaluation

Save model and generate classification reports

Model Compilation

```
model.compile(  
    optimizer="Adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Purpose

Configures the learning process using the Adam optimizer, integer target loss (sparse_categorical_crossentropy), and accuracy evaluation metrics.

`optimizer="Adam"`

Adaptive learning rate optimizer combining momentum and RMSProp for efficient gradient descent.

`loss="sparse_categorical_crossentropy"`

Suitable for integer-encoded class labels in multi-class classification tasks.

`metrics=["accuracy"]`

Tracks classification accuracy across training and validation sets during each epoch.

Training & Early Stopping Callback

EarlyStopping Configuration

```
# EarlyStopping configuration
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    verbose=1,
    mode="auto",
    restore_best_weights=True,
    start_from_epoch=3
)
```

01

monitor="val_loss"

Watches validation loss as the stopping criterion each epoch.

03

restore_best_weights=True

Automatically reverts model weights to the epoch with lowest validation loss.

Model Fitting

```
# Model Fitting
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=15,
    callbacks=[early_stopping]
)
```

02

patience=3

Tolerates up to 3 epochs without improvement before halting training.

04

start_from_epoch=3

Delays early stopping checks until epoch 3 to allow initial convergence.

❏ **Purpose:** Trains the classification network over 15 epochs while monitoring validation loss to prevent overfitting and restore optimal model weights automatically.

Model Export & Performance Evaluation

Model Saving & Downloading

```
# Model Saving and Downloading
model.save("model_2.keras")
print("Model saved successfully!")
files.download("model_2.keras")
print("Model downloaded successfully!")
```

`model.save()`

Persists the trained model as a `.keras` file for portability and reuse.

`classification_report()`

Generates precision, recall, and F1-score across all classes for both train and test subsets.

Training & Test Set Evaluation

```
# Training Set Evaluation
y_train_pred = model.predict(X_train, batch_size=32)
y_train_pred = np.argmax(y_train_pred, axis=1)
print(classification_report(y_train, y_train_pred))
del y_train_pred

# Test Set Evaluation
y_test_pred = model.predict(X_test, batch_size=32)
y_test_pred = np.argmax(y_test_pred, axis=1)
print(classification_report(y_test, y_test_pred))
```

`np.argmax()`

Converts softmax probability outputs to discrete integer class predictions.

`del y_train_pred`

Frees memory after training evaluation before running inference on the test set.

Purpose: Persists the trained `.keras` file locally, computes class predictions across training and test subsets, and generates complete evaluation reports (precision, recall, F1-score).